Análisis de Escalabilidad — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar hasta qué punto el sistema puede escalar (dispositivos, mesas, áreas) sin colapsar, y qué mecanismos se necesitan en cada nivel.

---

1. Escenarios de escala

Escenario A — Restaurante pequeño (actual)
Métrica	Valor
Dispositivos	2-5 (1 caja + 1-4 tablets/meseros)
Mesas	4-15
Áreas	1-2 (salón, cocina)
Comandas/hora	6-12
Usuarios concurrentes	2-5
Ancho de banda necesario	< 1 Mbps
Veredicto	✅ Funciona sin problemas

Escenario B — Restaurante mediano
Métrica	Valor
Dispositivos	6-15 (1 caja + 5-10 tablets + 2-4 cocina/bar)
Mesas	15-40
Áreas	2-4 (salón, cocina, bar, terraza)
Comandas/hora	30-80
Usuarios concurrentes	6-15
Ancho de banda necesario	2-5 Mbps
Veredicto	✅ Funciona con optimizaciones menores

Escenario C — Restaurante grande / cadena
Métrica	Valor
Dispositivos	16-40 (2-4 cajas + 10-30 tablets + 4-8 cocina/bar)
Mesas	40-120
Áreas	4-8 (múltiples salones, cocinas, bares)
Comandas/hora	100-300
Usuarios concurrentes	16-40
Ancho de banda necesario	10-30 Mbps
Veredicto	⚠️ Requiere cambios arquitectónicos

Escenario D — Evento masivo / catering
Métrica	Valor
Dispositivos	40-100+
Mesas	100-500+
Áreas	10+
Comandas/hora	500-2,000+
Usuarios concurrentes	40-100+
Ancho de banda necesario	50-200 Mbps
Veredicto	❌ Requiere arquitectura distribuida

---

2. Análisis por componente

2.1 mDNS (descubrimiento)

Escenario	Dispositivos	mDNS viable?	Limitación

A (pequeño)	2-5	✅	Ninguna
B (mediano)	6-15	✅	Ninguna
C (grande)	16-40	⚠️	Multicast puede degradar con 40+ peers
D (masivo)	40-100+	❌	mDNS no escala a 100+ (multicast storm)

Conclusión: mDNS funciona bien hasta ~40 dispositivos. Más allá, usar cloud discovery.

2.2 Servidor LAN (Rust, port 3000)

Escenario	Req/s estimado	Semaphore (64)	Veredicto
A	1-3 req/s	✅ Sobrado	OK
B	5-15 req/s	✅ OK	OK
C	15-40 req/s	⚠️ Ajustado	OK con stagger
D	40-100+ req/s	❌ Insuficiente	Requiere múltiples servidores

Cálculo: 40 dispositivos × 1 req cada 2s = 20 req/s. Semaphore de 64 → OK. Pero si todos sincronizan al mismo tiempo (thundering herd), puede saturar.

Fix: Stagger de polls (cada dispositivo con offset aleatorio de 0-2s).

2.3 WebSocket push (port 3001)

Escenario	Conexiones WS	Broadcast	Veredicto
A	2-5	✅	OK
B	6-15	✅	OK
C	16-40	⚠️	OK (broadcast O(n))
D	40-100+	❌	Broadcast O(n²) → colapso

Cálculo: Con 40 dispositivos, cada broadcast envía 40 mensajes. Si 10 comandas/min → 400 mensajes/min → 6.7 msg/s. OK.

Con 100 dispositivos: 100 × 10 = 1,000 msg/min → 16.7 msg/s. Puede degradar.

2.4 Sync runtime (estado de mesas)

Escenario	Mesas	Tamaño sync	Frecuencia	Bytes/s	WiFi LAN
A	4-15	0.5-2 KB	2-4s	0.1-1 KB/s	✅
B	15-40	2-5 KB	2-4s	0.5-2.5 KB/s	✅
C	40-120	5-15 KB	2-4s	1.25-7.5 KB/s	✅
D	100-500+	15-60 KB	2-4s	3.75-30 KB/s	⚠️

Problema: Sync runtime envía el estado COMPLETO de todas las mesas cada 2-4s. Con 120 mesas, cada sync es ~15 KB. × 40 dispositivos = 600 KB/s de tráfico. WiFi puede manejarlo, pero es ineficiente.

Fix: Sync incremental (solo cambios desde lastSync, no estado completo).

2.5 Cloud sync (Supabase)

Escenario	Req/min	Supabase free tier	Veredicto
A	2-5	✅ Sobrado	OK
B	5-15	✅ OK	OK
C	15-40	⚠️ Ajustado	OK con batching
D	40-100+	❌ Insuficiente	Requiere plan pago + batching

Supabase free tier: 500 MB DB, 1 GB storage, 2 GB bandwidth/mes, 50,000 MAU.

Con 40 dispositivos × 1 sync cada 4s = 10 req/s = 600 req/min = 864,000 req/día. Supabase puede manejarlo pero el bandwidth puede ser un problema (1.8-14.4 MB/hora × 24h = 43-345 MB/día).

2.6 Persistencia local

Escenario	Datos locales	localStorage (4.5MB)	SQLite (Tauri)
A	~1-2 MB	✅ OK	✅ OK
B	~3-8 MB	⚠️ Ajustado	✅ OK
C	~10-30 MB	❌ Insuficiente	✅ OK
D	~50-200 MB	❌ Insuficiente	✅ OK (con índices)

Problema: localStorage tiene tope de 4.5MB. Con 120 mesas + comandas + runtime, se excede.

Fix: Migrar de localStorage a SQLite (Tauri) o IndexedDB (web) para datos grandes.

2.7 mDNS (descubrimiento)

Escenario	Peers descubiertos	mDNS viable?
A	2-5	✅
B	6-15	✅
C	16-40	⚠️ (multicast puede degradar)
D	40-100+	❌ (multicast storm)

Fix para C/D: Usar cloud discovery (Supabase) en lugar de mDNS para >40 dispositivos.

---

3. Cuellos de botella por escenario

Escenario A (pequeño) — 2-5 dispositivos, 4-15 mesas
```
Cuellos de botella: NINGUNO
Todo funciona sin problemas.
```

Escenario B (mediano) — 6-15 dispositivos, 15-40 mesas
```
Cuellos de botella:
  1. Sync runtime completo cada 2-4s (2-5 KB × 15 dispositivos = 30-75 KB/s)
  2. localStorage puede acercarse al límite (3-8 MB)
  3. mDNS puede degradar ligeramente con 15 peers

Fixes:
  - Sync incremental (solo cambios)
  - Migrar runtime a SQLite/IndexedDB
  - Stagger de polls
```

Escenario C (grande) — 16-40 dispositivos, 40-120 mesas
```
Cuellos de botella:
  1. Sync runtime completo (5-15 KB × 40 = 200-600 KB/s) ← CRÍTICO
  2. localStorage insuficiente (10-30 MB)
  3. mDNS degradado (16-40 peers)
  4. Supabase bandwidth (43-345 MB/día)
  5. Semaphore de 64 conexiones (puede saturar con thundering herd)
  6. WS broadcast O(n) (40 × 10 comandas/min = 400 msg/min)

Fixes necesarios:
```

```
  - Sync incremental (CRÍTICO)
  - Migrar a SQLite/IndexedDB (CRÍTICO)
  - Cloud discovery en lugar de mDNS para >20 dispositivos
  - Batching de sync (enviar cada 5-10s en lugar de 2-4s)
  - Stagger de polls con jitter (0-2s aleatorio)
  - Aumentar semaphore a 128-256
```

Escenario D (masivo) — 40-100+ dispositivos, 100-500+ mesas
```
Cuellos de botella:
  1. TODO lo de Escenario C × 2-3
  2. mDNS colapso (multicast storm)
  3. WS broadcast O(n²) → 100 × 100 = 10,000 msg/min
  4. Supabase rate limits
  5. Un solo servidor LAN (port 3000) como SPOF

Fixes necesarios (arquitectura distribuida):
  - Múltiples servidores LAN (sharding por área)
  - Cloud como backbone principal (no LAN)
  - WS con rooms (solo broadcast a dispositivos relevantes)
  - Sync incremental + delta compression
  - Caché local con invalidación
  - Load balancer para múltiples cajas
```

---

4. ¿Puede colapsar? Sí, en estos puntos

Punto de colapso	Escenario	Causa	Consecuencia
localStorage lleno	C (grande)	4.5MB tope	Datos no se guardan, pérdida de información
mDNS multicast storm	D (masivo)	40+ peers en multicast	Descubrimiento falla, dispositivos no se encuentran
Semaphore saturado	C (grande)	Thundering herd (todos sincronizan al mismo tiempo)	Requests se rechazan, sync falla
WS broadcast O(n²)	D (masivo)	100 dispositivos × 100 mensajes	Latencia alta, tablets se congelan
Supabase bandwidth	C/D	43-345 MB/día	Throttling, sync lento
Sync runtime completo	C (grande)	5-15 KB × 40 dispositivos = 200-600 KB/s	WiFi puede manejarlo pero es ineficiente

---

5. Roadmap de escalabilidad

Nivel 1 — Pequeño (ya funciona)
- 2-5 dispositivos, 4-15 mesas, 1-2 áreas
- **No requiere cambios**

Nivel 2 — Mediano (optimizaciones menores)
- 6-15 dispositivos, 15-40 mesas, 2-4 áreas
- **Cambios**: Sync incremental, migrar runtime a SQLite, stagger de polls
- **Esfuerzo**: 1-2 semanas

Nivel 3 — Grande (cambios arquitectónicos)
- 16-40 dispositivos, 40-120 mesas, 4-8 áreas
- **Cambios**: Cloud discovery, sync incremental + delta, batching, SQLite obligatorio, semaphore 128
- **Esfuerzo**: 3-4 semanas

Nivel 4 — Masivo (arquitectura distribuida)
- 40-100+ dispositivos, 100-500+ mesas
- **Cambios**: Múltiples servidores LAN, sharding por área, WS rooms, cloud backbone, load balancer
- **Esfuerzo**: 2-3 meses

---

6. Resumen ejecutivo

Pregunta	Respuesta
¿Funciona con 2-5 dispositivos y 4-15 mesas?	✅ Sí, sin problemas
¿Funciona con 6-15 dispositivos y 15-40 mesas?	✅ Sí, con optimizaciones menores
¿Funciona con 16-40 dispositivos y 40-120 mesas?	⚠️ Requiere cambios arquitectónicos (sync incremental, SQLite)
¿Funciona con 40-100+ dispositivos y 100-500+ mesas?	❌ Requiere arquitectura distribuida
¿Colapsa a mitad del trabajo?	No en niveles 1-2. En nivel 3 sin optimizaciones, sí
¿Bluetooth funciona?	Solo para comandas/presencia en nivel 1-2 (3-5 dispositivos)
¿Internet es mejor?	Sí, para nivel 3+ usar cloud como backbone

Lo que hay que hacer para llegar a nivel 3 (grande)
1. **Sync incremental** (solo cambios, no estado completo) — CRÍTICO
2. **Migrar runtime de localStorage a SQLite/IndexedDB** — CRÍTICO
3. **Cloud discovery** en lugar de mDNS para >20 dispositivos
4. **Batching** de sync (cada 5-10s en lugar de 2-4s)
5. **Stagger de polls** con jitter (0-2s aleatorio)
6. **Aumentar semaphore** a 128-256
7. **WS rooms** (broadcast solo a dispositivos relevantes)

Lo que hay que hacer para llegar a nivel 4 (masivo)
1. **Todo lo de nivel 3** +
2. **Múltiples servidores LAN** (sharding por área)
3. **Cloud como backbone principal** (no LAN)
4. **Load balancer** para múltiples cajas
5. **Caché local** con invalidación

---

7. Conclusión

El sistema **funciona bien para restaurantes pequeños y medianos** (hasta ~15 dispositivos y ~40 mesas). Para llegar a **restaurantes grandes** (40+ dispositivos, 120+ mesas), se necesitan cambios arquitectónicos (sync incremental, SQLite, cloud discovery). Para **eventos masivos** (100+ dispositivos), se necesita una arquitectura distribuida.

No colapsa a mitad del trabajo si se implementan las optimizaciones del nivel 2-3. Sin ellas, con 40+ dispositivos y 120+ mesas, el sync runtime completo cada 2-4s saturará el ancho de banda y el localStorage se llenará.

Bluetooth es viable solo como fallback para comandas/presencia en restaurantes pequeños (3-5 dispositivos). Para todo lo demás, WiFi LAN + Cloud es la estrategia correcta.